

AGILIC

GETTING STARTED GUIDE

Setting Up Your Organization in Agilic

A plain-English guide to Item States, WP Phases and Labels

Who it is for	Anyone setting up a new organization within Agilic. No project-management background needed.
How long	About 20 minutes to read through. About 30-60 mins to read and do the exercises. Appendices are extra detail to help with understanding.
What you will produce	A state list, a phase list, a team list, a label list, and a named owner.
Version	v3.4 — September 2026

Contents

Before You Start

- Three words to know before you start
- The three questions
- How this guide is organized

Step 1 — Write your Item States

- A starting set that works for almost anyone
- The Status field — the one that drops unnecessary meetings

Step 2 — Write your WP Phases

- What this looks like in practice
- Why Phases overlap, and why that is useful
- How to find your Phases in ten minutes or less
- A starting set that works for many — but not everyone
- Does this work for Agile as well as waterfall?
- The exception — using Phases to move work
- Phases are per-WP, but should mostly match

Step 3 — Choose your Teams

Step 4 — Choose your Labels

- The three kinds of labels, in plain words
- Two questions before you create any label
- A starting set
- A Note about Items and some labels to avoid

Step 5 — Decide who owns the list

- The minimum set of rules for organization-wide label management

Step 6 — Test it before you roll it out

- The ten-item test
- What good looks like

The Five Mistakes That Kill These Setups

Appendix A — Why State and Status Are Two Fields

- How to make it stick

Appendix B — Designing Phases Properly

- What a Phase actually is
- How Phases behave on a timeline
- Finding the parallel Phase
- When to add a Phase
- Using Phases to move work — the legitimate exception

Appendix C — Getting Label Types Right

- Simple labels
- Unbound labels — tick as many as apply
- Bound labels — pick exactly one
- Naming conventions worth adopting

Appendix D — Preventing Label Sprawl

- How bad it gets
- What actually works

Appendix E — Three Worked Examples

- E.1 A commercial fit-out contractor
- E.2 A regional food-bank charity
- E.3 A software company with an infrastructure build

Appendix F — One-Page Checklist

Appendix G — Terminology and Additional Training

- G.1 How work is structured
- G.2 Who does what
- G.3 How work is classified
- G.4 Flags and views
- G.5 Additional Training

Before You Start

Setting up a new organization in Agilic means making a handful of decisions about vocabulary: what the buttons say, what the columns are called, and how work gets sorted. Those decisions look small, but they can have a big impact. Get them right and every board, report and search works the way people expect. Get them wrong and you spend the next two years arguing about what "In Progress" means.

This guide takes about twenty minutes to read and walks you through six steps. You do not need to be a project manager. You do not need to know any methodology. You need to know how work actually gets done in your organization, and you need to be willing to write down the shortest honest answer.

The one idea behind all of this

Every field you set up should answer a question that no other field already answers. That is the whole principle. When two fields answer the same question, they eventually disagree, and once they disagree nobody trusts either one. Every rule in this guide comes back to that.

Three words to know before you start

Agilic uses three terms constantly, and very little below will make sense without them. Appendix G has a short glossary and links to the general training videos and Training Documents Wiki.

Work Package (WP)

A body of work. Any size. Period. A Work Package can be a large multi-team project or one person's small task — it is whatever size body of work you need to manage. Every WP has an Objective, an Owner who is accountable for its success, and a Driver who is responsible for delivering it. Where this guide says Effort, it means the whole body of work a WP represents.

Items and the 4 D's

An Item is the lowest level of tracking in Agilic — an activity, a task, a list entry, a user story, a milestone. Every Item sits inside one of four Work Item Types, known as the 4 D's:

Work Item Type	The work in it
DEFINE	Working out what the thing actually is — What do we need to know to successfully Deliver the WP Objective? Requirements, design, plan, brief.
DO WORK	What work needs accomplished to Deliver the WP Objective?
DOCUMENT	Documentation, recordings etc needed to support and Deliver the WP Objective, so someone who was not there can understand it.
DELIVER	The official (and unofficial) final deliverables needed for saying we accomplished this WP Objective.

Every Item can have one or more people Assigned, meaning officially working on it, and only one person can be made Responsible, meaning the point of contact for it.

RIDE

RIDE is a fifth Work Item Type alongside the 4 D's. These are the things that pop up day to day and are often what derail projects. Risks, Issues, Dependencies and Escalations — it does not matter what you call them, the pattern is nearly always the same: you have to document it, show what Items are impacted in the Work Package, then obtain something from someone, usually from outside the Effort - information, confirmation, awareness or a decision.

The three questions

Agilic gives you four main places to record something about an Item. Each one answers a different question. Learn these four and the rest of the guide is detail.

Field	The question it answers	In plain words
Item State	Where is this right now?	Has anyone started it? Is it stuck? Is it finished?
Item Status	WHY is the Item in the State it is right now?	WHY is this Blocked? You've been In Progress for 3 weeks on a two day activity - WHY?
WP Phase	What part of the Effort does this belong to?	Is this planning work, delivery work, or the reporting that runs alongside both?
Labels	What else do I need to know about it?	Everything that is neither of the above — how urgent, which function needs to know, which product, which client.

All three can be Kanban columns

Item States, Phases and Labels can each be used to build the columns on a Kanban board — as can the Milestone and Show on Roadmap flags. That is worth knowing before you design any of them, because it means each of these lists is a possible way to lay out a board. It also means a badly designed list produces a badly designed board for everyone who groups by it. Every card on the Kanban can show the most recent Status, and the Status can be updated easily on the Kanban without fully opening the Item. If you are unfamiliar with Kanban boards, please check Agilic's Training Documents Wiki for additional information.

How this guide is organized

The six steps below are the whole job. Each one ends with a short decision you write down. Appendices at the back go deeper on anything you want to understand properly — they are optional, and nothing in the six steps depends on reading them.

Appendix	Read it if you want to...
A	Understand why State and Status are separate fields, and what happens when people stop filling one in.
B	Design Phases properly — including why they overlap, how they behave on a Gantt chart, and when to add one.
C	Understand the label types and how to use them, especially the difference between the two-part kinds.
D	Stop your label list growing out of control.
E	See three complete worked examples from different industries.
F	Print a one-page checklist and work through it with your team.
G	Look up any term used in this guide, and find the training videos and Training Documents Wiki.

STEP 1 Write your Item States

An Item State answers one question: where is this piece of work right now? Not what kind of work it is. Not how important it is. Just — has anyone started, is it stuck, is it done.

This list should be short and it should be the same everywhere in your organization. The temptation is to make it longer so it says more. Resist it. Every state you add is another thing every person has to understand, and another column on every board that groups by State.

A starting set that works for almost anyone

Agilic has a System Default Template that is a good, generic set of Item States. It has been tested against software, marketing, construction, compliance and fundraising work, and it holds up in all of them; however they may still not work for your organization.

State	Use it when	Why it earns its place
Not Started	The work is agreed but nobody has begun.	Distinguishes "waiting" from "nobody has looked at this."
In Progress	Someone is actively working on it now.	The one everyone expects.
Blocked	Something outside your control is stopping progress.	This is the state that makes your reports honest. Without it, stuck work hides inside In Progress for months.
On Hold	You chose to pause it. Nothing is in the way.	Different from Blocked. Blocked is "we can't." On Hold is "we decided not to." They need very different responses.
Complete	Done, and whatever counts as finished has happened.	Self-explanatory, but agree what "finished" means before you launch.
Cancelled	It will not be done, and that is final.	Keeps the record. Never delete work — you lose the reason it was dropped.

The Status field — the one that drops unnecessary meetings

State says where something is. Status is a short line of free text saying WHY. "Blocked" tells you nothing useful on its own. "Blocked — waiting on signed contract from the council, last chased 12 March" tells you everything.

Make Status mandatory in your team's working habits for three states: Blocked, On Hold and Cancelled. Those are the three where the state alone leaves an obvious unanswered question. For the others it is optional.

Be aware that this only works if people actually do it. This pattern exists in other tools and is the most commonly abandoned convention in project setup — see Appendix A for why and what to do about it.

Write this down

Your Item State list, and which states require a Status note. If you copied the six above, you are done with Step 1.

STEP 2 Write your WP Phases

A WP Phase answers a different question: what are the standard Project Phases across this entire Effort? Where State asks whether an Item is moving, Phase asks what part of the Effort this Item belongs to.

The standard Work Item Types — Define, Do Work, Document and Deliver — are pre-defined for you and already cover the basic shape of any task. That is deliberate: it frees WP Phases up to be something more meaningful to your organization rather than a generic set you would never have chosen. Many reports in Agilic allow you to display Items by Work Item Type or by Phase.

What this looks like in practice

There are two styles of Phases you might consider setting up

1. Phases as a Stage Control: Initiation, Planning, Execution, Closure - each of these Phases is, more or less, a Stage Gate. They may have some overlap but in general you will be exiting one Phase and moving into the next Phase.
2. Phases as a Process Control: Monitoring & Control - These would be activities that are needed in every (or most) Phases and would span multiple Stage Phases.

Say the Effort is opening a new store. It contains Items like these:

An Item in the Effort	The part of the Effort it belongs to
Choose the site	INITIATION
Sign the lease	INITIATION
Hire and schedule staff	PLANNING
Order shelving and fixtures	PLANNING
Fit out the space	EXECUTION
Run the launch promotion	EXECUTION
Weekly budget report to the owner	MONITOR & CONTROL
Monthly progress review	MONITOR & CONTROL
Final accounts and lessons learned	CLOSING

Look at the two highlighted rows. The weekly budget report does not start in Initiation, graduate to Planning, and end up in Execution. It belongs to Monitor & Control from the day it is created until the store opens. That is the difference between a Phase as a Stage Control and a Phase as a Process Control.

Items normally belong to one Phase for their whole life

The Item State changes — Not Started, In Progress, Complete. The Phase usually does not. A task to order shelving is a Planning task on the day it is created and still a Planning task on the day it is finished. There is an exception, covered below, but treat “belonging” as the normal case.

Why Phases overlap, and why that is useful

Because Items belong to Phases rather than passing through them, more than one Phase is normally live at the same time. Monitor & Control is the clearest case — in the default set it runs alongside everything else from start to finish. A model where work moves through one stage at a time simply cannot express that.

Phases carry no dates of their own. A Phase stretches from the earliest start date of the Items inside it to the latest due date, so the bands on a Gantt chart shift as your work shifts:

- If your phases are meant to run in parallel, you will see them running in parallel, which is simply the truth.
- If your phases are meant to be sequential and two of them start to overlap, you are watching schedule creep appear — usually weeks before anyone would have reported it.

How to find your Phases in ten minutes or less

1. Pick one typical Effort your organization runs — a project, a campaign, a build, a client engagement.

2. Write down fifteen to twenty real Items that Effort would contain. Actual tasks, in your own words. Do not tidy them up.
3. Sort them into piles based on what part of the Effort each one belongs to. Ignore timing and ignore order — you are grouping by kind, not by sequence.
4. Name each pile. Those names are your Phases.
5. Now look for the pile that runs the whole way through — reporting, governance, quality checks, safety, client communication. Most organizations have one and most have never named it. That is your parallel Phase and it is usually the most valuable one you will find.
6. Aim for three to eight piles. More than eight and you have sorted by task type rather than by part of the Effort. Fewer than three and one pile is hiding something.

A starting set that works for many — but not everyone

If the ten-minute exercise gives you nothing better, Agilic ships a System Default set of WP Phases following PMI's process groups. Four run broadly in sequence and the fifth runs alongside all of them.

#	Phase	What is happening	It leaves when...
1	INITIATION	This Effort is being defined and agreed to.	It has been approved to begin, with the right people in place to fully estimate the Items on the Effort.
2	PLANNING	The correct people are part of the Effort in order to get a sufficient plan together and sufficiently estimated.	You have enough information to begin work. This does not always mean planning is finished — that depends on your company and industry.
3	EXECUTION	Items that deliver something of value against the scope of the Effort.	All the value of this Effort has been delivered.
4	MONITOR & CONTROL	All the administrative work that happens across the life cycle of the Effort — reporting, change control, risk review, governance. Runs alongside the other phases rather than after them.	Either Execution has completed, or the Effort has finished Closing. Which one depends on your company culture.
5	CLOSING	Everything needed to make sure the Effort is fully shut down.	Nothing — this is the end.

Does this work for Agile as well as waterfall?

Yes, and the reason is that Phases describe the Effort, not the cadence. In an Agile Effort, Initiation and Planning still happen — they are just shorter and revisited often. Most delivery Items belong to Execution regardless of which sprint they land in, because sprint membership is carried by sprints, not by Phases. Monitoring & Control covers the ceremonies, the reporting and the change control that surround the sprints.

The instinct you should resist is creating a Phase per sprint or per iteration. That produces dozens of phases describing time rather than substance, and it makes the Gantt difficult to read. Cadence is not structure.

It is important to note that Agilic has an independent Work Breakdown Structure (WBS) separate from WP Phases. You can align the WBS with WP Phases or not — both options can be displayed on the Gantt. If you are unfamiliar with what a WBS is, please check Agilic's Training Documents Wiki.

The exception — using Phases to move work

Some kinds of work genuinely do travel. An IT Operations ticket might move from Triage to Diagnosis to Resolution to Review, where each step is a real change in who is handling it and what is being done.

If your work looks like that, Phases can carry that movement alongside Item State, and this is a legitimate use.

Two cautions. Do not mix the two styles of Phase patterns inside a single WP, because a board grouped by Phase would then show as belonging to a Content Grouping for some Items and progress for others, and nobody can read that. Also, check first that Item State is not already enough — if the only difference between your Item States and proposed phases is whether someone has started, then you have described States twice.

Not every Work Package needs Phases

Phases exist to divide an Effort into parts and make it more manageable. If Phases feel forced, leave them alone and let Item State and Labels do the work. They may be useful later or, by not using them, you may have saved yourself some work.

Phases are per-WP, but should mostly match

Each Work Package can have its own Phases. That flexibility is useful, but it is also a double-edged sword. If every WP has its own unique set of Phases, then they are no longer useful in reporting across the company.

The practical answer is a standard set used by every WP doing a similar type of work, with a small number of additions where a WP genuinely needs one. If your compliance team needs an AUDIT phase that means nothing to anyone else, give it to them. If three WPs have all invented their own version of roughly the same phase, that is a sign the standard set is missing something — fix the standard set instead.

Write this down

Your standard Phase list, which one runs in parallel with the others, and any known WP that needs an extra Phase and what it is called.

STEP 3 Choose your Teams

Agilic can handle any number of functional Teams — Networking, Marketing, Sales, Development, Operations, Electrical, and so on. You are not required to use Teams, but it is often useful for people with a similar kind of role, reporting to the same manager, to sit on the same Team.

Teams, in Agilic, are based on peers working on the same Team and reporting to the same Manager. The everyday payoff is coverage. When someone is out sick, it is easy to find the work they were carrying and see who could pick it up.

For more information on setting up Teams, check Agilic's Training Documents Wiki.

Teams and a Team label are not the same thing

Individuals are part of a Team. Team Views can see all the work Team Members are Assigned to. The Aware : [Team Name] label in Step 4 records which functional Teams need to be AWARE of a piece of work, which is often a wider group. A security review might be assigned to one Team and need three others watching it. Keep the distinction clear when you write your definitions, because this is the one place in the setup where two things share a name.

Write this down

Your Team list, or a note that you are not using Teams for now. This one is easy to add later.

STEP 4 Choose your Labels

Labels hold everything that is not State and not Phase. How urgent it is, which functions need awareness, which product it touches, which client it is for, which regulation it satisfies.

Labels are also where organizations lose control, because they are the easiest thing to create and the hardest thing to clean up. Step 5 deals with that. This step is about picking the right ones and the right kind.

The three kinds of labels, in plain words

Kind	Think of it as	How many per item	Example
Simple	A sticker or category	Any number of different stickers or categories	Quick Win #Legal
Unbound	A tick-box list — tick as many as apply	Several from the same list	Team : Sales Team : Marketing
Bound	A drop-down — pick exactly one	One from the list, no more	Priority :: High Priority :: Low

Choosing between Unbound and Bound is the only part people get wrong. The test is one question: could a single item ever honestly need two values from this list at the same time?

- Priority — nothing is both High and Low. Bound.
- Team — a piece of work can easily involve Sales and Marketing both. Unbound.
- Client — usually one, but sometimes a shared project. Ask before you decide.

If you force a Bound label onto something that genuinely needs two values displayed, people will generally not complain. They will just pick one and drop the other, and you will never know the data is wrong. That is why this one matters more than it looks.

Two questions before you create any label

1. Does another field already answer this? Since Agilic has a Due Date field, do not create a "Due This Month" label. It will drift out of sync the first time a date changes.
2. Will anyone actually filter or report on this? A label nobody searches by or needs to track is pure typing. If you cannot name the report it feeds, do not create it.

A starting set

Most organizations need some version of these. Adapt the values, keep the shape. Agilic does not pre-define any labels of any kind.

Label	Kind	What it does
Priority :: [Level]	Bound	Critical / High / Medium / Low. Agree in advance roughly what share of work may be Critical, or the word stops meaning anything.
Effort :: [Estimate]	Bound	Small / Medium / Large, or T-shirt sizes. Deliberately rough. This is for planning conversations, not for estimating.
Importance :: [Level]	Bound	High / Medium / Low / Parking Lot
Priority :: [Level]	Bound	1 / 2 / 3 / 4 / 5 / Medium / Low — An Item can't be Priority 1 and Medium.

Label	Kind	What it does
Urgency :: [Sequence]	Bound	Now / Next / Later. Lets you sequence work without committing to dates — useful when you have to show a plan to someone outside the organization.
Area :: [Reason]	Bound	Why you are doing it — Revenue, Compliance, Efficiency, and so on. Pick the strongest single reason. Makes prioritization reviews possible without reading every description.
Aware : [Team Name]	Unbound	Which functions need to be AWARE of this work. Not who it is assigned to — that is the Team structure from Step 3. Often a wider group than the one doing the work.
Blocked External	Simple	Optional. Some organizations prefer a flag to spot outside-dependency work quickly, alongside the Blocked state.

A Note about Items and some labels to avoid

1. All standard Items (Define, Do Work, Document, Deliver) and RIDEs (Risks, Issues, Dependencies and Escalations) have the ability to be turned into Milestones.
2. All Milestones and RIDEs have the ability to be marked as Escalated or Critical.
3. This information is built into Agilic already, so it does not need a label. For more information, check Agilic's Training Documents Wiki.

Organization-level and Work Package-level labels

The common labels you need across most of your Efforts should be created at the Organization level, in Org Admin Settings. Every Work Package can then use them. Each Work Package can also create labels of its own for needs unique to that Effort. Because only users with Org Admin permission can add organization-wide labels, this arrangement keeps sprawl to a minimum while still letting local needs be met locally.

Write this down

Your common label list that will handle most of your Item labeling needs, with the kind (Simple, Unbound or Bound) marked and the values listed for each of the two-part labels (Unbound / Bound).

STEP 5 Decide who owns the list

This step takes five minutes and prevents the single most common failure in this whole exercise.

If you have several Org Admin users who can all create labels, then as people come to them with a need, new labels will get added without much deliberate thought. Not maliciously — just because a project or team leader would rather not add the same label to a second Effort later. But that is not the same as the label being valuable to the organization as a whole.

This is not hypothetical

Consultants who clean up these systems report a nonprofit whose tag list passed 2,000 entries, and a financial technology firm whose contact-reason drop-down reached over 1,400 options, mostly duplicates. Nobody set out to do that. It happens one convenient new label at a time. See Appendix D.

The minimum set of rules for organization-wide label management

Rule	Why
One named person owns the organization-wide list. Everyone else requests.	Someone has to be able to say "we already have one of those." A group cannot do this; an individual can.
A WP owner may add labels for their own WP but not organization-wide ones.	Local needs are real. Let them be met locally without changing the vocabulary for everyone. Organization-wide labels require Org Admin permission.
If three separate WPs invent roughly the same label, the owners should request it be added to the standard set, and retire the copies.	This is how you find out what the standard set was missing, without guessing up front.
Start strict. Loosen later if it is genuinely too tight.	Tightening a loose system means taking things away from people, which is hard. Loosening a strict one is easy and popular.

Write this down

Who owns the list, and what someone does when they want a new label.

STEP 6 Test it before you roll it out

Do not launch this to your organization on the strength of it looking sensible on paper. It takes half an hour to find out whether it actually works.

The ten-item test

1. Pick ten real pieces of work from the last few months. Deliberately choose awkward ones — the thing that got cancelled, the thing that dragged for months, the thing two teams argued over.
2. For each one, fill in the State, the Phase and every label, as it would have been at its most confusing moment.
3. Note every time you hesitate, have to look something up, or find yourself wanting a value that does not exist.
4. Now hand the same ten items to somebody who was not in the room when you designed this, and have them do it independently.
5. Compare. Every disagreement is a definition that is not clear enough yet.

Disagreements are the point of the exercise. If two sensible people classify the same work differently, your live data will be inconsistent in exactly that way, at scale, forever. Fix the wording now while it costs nothing.

What good looks like

- Both people picked the same State on at least nine of ten.
- Nobody needed a value (Label, State, and/or Phase) that did not exist more than once or twice.
- You can explain any state or phase to a new starter in one sentence, without saying "it depends."

And then leave it alone for a quarter

Resist changing anything for the first three months except outright mistakes or missed critical needs. Every change costs everyone a small amount of relearning, and most early change requests are people asking for their old system back rather than the new one being wrong. After a quarter you will have real evidence about what is actually missing.

The Five Mistakes That Kill These Setups

Every one of these has a reasonable-sounding argument behind it. That is why they are common.

Mistake	What it looks like	What to do instead
Too many states	Fifteen states because every team wanted their own. Boards become unreadable and nobody knows which of three "waiting" states to use.	Eight or fewer, shared by everyone. Six or fewer is better if you can do it.
Work types used as states	Things like In Design, In Testing, Awaiting Legal.	Move them to Phases (Design, Testing, Legal). The state should only say whether the Item is moving.
Two fields answering one question	A "Status :: xxxx" label alongside the State field.	Define more descriptive Item States. Keep in mind the "Too many states" mistake above.
	Two labels handling the same thing, often because of a spelling difference.	Pick one and delete the other. If they exist as custom labels in different Work Packages, ask for a common label to be added across the Organization so it cannot happen again.
Labels for things that are already fields	A "Due Soon" label, an "Owner: Priya" label, a "Milestone" label.	Use the real field. Labels that mirror fields go stale silently.
Designing for the exception	A whole extra state for a situation that came up twice last year.	Handle rare cases in the Status note. Fields are for the common case.

If you only remember one thing from this guide, make it the highlighted rows. Two fields answering the same question is the failure that quietly ruins the data, because unlike the others it does not look like a problem until someone asks a question the reports cannot answer.

Appendix A — Why State and Status Are Two Fields

The short version: State is a fixed list you pick from, Status is a sentence you write. State makes reports possible. Status makes them useful.

A report saying eleven items are Blocked is an alarm with no information — requiring extensive follow-up and potential meetings. A report saying eleven items are Blocked and listing eleven one-line reasons is something a manager can act on in five minutes. The first is a number, the second is a to-do list.

How to make it stick

- Require it for three states only — Blocked, On Hold, Cancelled. Requiring it everywhere guarantees people type "ok" to get past it.
- Make reading them a habit before making writing them a rule. If Status notes are visibly used in a weekly review, people write them. If they disappear into a database, they stop within a month.
- Keep them short and dated. "Waiting on supplier quote, chased 14 March" is a good one. A paragraph is not. Note that Status in Agilic is automatically timestamped for when the Status was entered, along with who made it. The most recent Status is always displayed on cards.
- If your team is not filling them in after a quarter, there is some kind of issue. Do not just keep asking — follow up with your team:
 1. Do they understand the value of taking 30 seconds to update the Status? It saves them answering emails, being caught for drive-by status questions, and reduces the number of meetings needed.
 2. Do they understand how easy it actually is to make the Status update?
 3. If yes to both, is the requirement of when a Status is needed appropriate? Did you set too strict a rule about what has to be entered, or can they just give the status in their own words?

The general wisdom is that using Status keeps everyone on the same page without requiring an in-person meeting or a current email. Status shows not only what the explanation is, but also who said it, and when they said it.

Appendix B — Designing Phases Properly

What a Phase actually is

Phases in Agilic are optional. A Phase is one of the major parts an entire Effort can be divided into. Items are attached under the Phase they belong to, and most Items stay there for their whole life. The Item State moves; the Phase generally does not.

This is worth stating plainly — under the model Agilic uses, multiple Phases can potentially be active at once.

How Phases behave on a timeline

In Agilic, Phases have no dates of their own. A Phase band on a Gantt chart runs from the earliest start date of the Items inside it to the latest due date, and it moves as those Items move. Three consequences follow:

- Phases holding undated Items contribute nothing to the chart, so you are never forced to invent a date for work nobody has committed to.
- Phases overlap and for a parallel phase, such as Monitoring & Control, that overlap is the whole point — it should span everything.
- Where phases are meant to run in sequence, watching two bands start to overlap is one of the earliest warnings of schedule creep you will get. It shows up in the chart before it shows up in anyone's report.

Finding the parallel Phase

Most organizations have one parallel Phase and most have never named it. It is the work that runs the whole way through and belongs to no single part: status reporting, governance, risk review, quality assurance, safety, client communication, change control.

Before it is named, this work is often invisible. It gets absorbed into whichever delivery phase it happens to touch, which makes that phase look inexplicably expensive and makes the reporting itself look like it costs nothing. Naming it is usually the single most valuable change an organization makes to its phase list, because it is the first time anyone can see how much of the week it actually consumes.

When to add a Phase

Add one when there are multiple parts of the Effort that genuinely should be grouped together and you can collectively say "we are done with this part." One classic case is an externally-controlled wait — an audit, a contract approval, a funder's decision. Between "we submitted it" and "they said yes" there is often months of real activity: answering questions, resubmitting, chasing.

Using Phases to move work — the legitimate exception

Some work genuinely travels. A support ticket that moves from Triage to Diagnosis to Resolution to Review is changing hands and changing character at each step, and Phases can carry that alongside Item State.

Two rules if you do this. Do not mix Phase patterns inside one Work Package, because a board grouped by Phase would then show belonging for some Items and progress for others, and it becomes unreadable. And confirm Item State is not already sufficient — if the only real difference between your proposed phases is whether someone has picked the work up, you have written States twice under another name.

Keeping phases comparable across Work Packages

The tension is real: local phases are more useful locally, shared phases are the only way to compare anything across the organization. The workable compromise is a standard set used by every WP doing a similar kind of work, plus rare and justified additions. If three WPs have each invented their own version of the same missing phase, do not approve three — add one to the standard set and retire the local copies.

Appendix C — Getting Label Types Right

Simple labels

One word or two — simple, straightforward categories or stickers. "Quick Win" is a Simple label; there is no "Slow Loss" to pair it with. If you find yourself wanting to create both a label and its opposite, you probably want a two-part label with two values.

Simple labels are the easiest to create and therefore the fastest to become meaningless. A useful discipline: a Simple label should change what somebody does. If seeing it on an item does not change anyone's behavior, it is decoration.

Unbound labels — tick as many as apply

Written as a category and a value with a single colon. An item can carry several values from the same category at once. Use them wherever an item can honestly belong to more than one value: functions that need awareness, products touched, regions affected, regulations satisfied.

The reason this matters is reuse. If one piece of compliance work impacts four different standards, an Unbound label makes that visible, and visible reuse is the difference between doing the work once and doing it four times.

Bound labels — pick exactly one

Written with a double colon, and only one value is permitted. Use Bound labels where two values at once would be a contradiction: priority, size, funding round, funnel stage.

The failure mode to watch for

When a Bound label is wrong for a dimension, users do not report it. They pick one value, silently drop the other, and move on. The data looks complete but is not. So the question to ask is never "is one value usually enough?" but "could one item ever honestly need two?" If yes, make it Unbound even if the two-value case is rare.

Naming conventions worth adopting

Convention	Reason
Category names are singular — Team, not Teams.	You are naming the dimension, not the collection. Mixed plurals produce near-duplicate categories.
Consistent capitalization, decided once.	This is the single largest source of accidental duplicates in every system that allows free typing. Agilic Labels specifically, will not allow duplicate text regardless of capitalization.
No abbreviations that are not already spoken aloud in your organization.	A label that can be misinterpreted by different groups is a label that is not applied consistently.
Keep Simple labels to one or two words.	They compete for space on the card. Labels are quick references and should not require extensive reading to know what they mean.
Never encode two facts in one label.	A label like "Urgent-Marketing" cannot be filtered on either fact. Two labels, always.

Appendix D — Preventing Label Sprawl

Every organization believes this will not happen to them. The published evidence from people who clean these systems up for a living says otherwise.

How bad it gets

A consultancy company specializing in taxonomy work described a nonprofit client where adding tags was easy and no review existed — the list eventually passed 2,000 entries. At a large financial technology organization, a drop-down of customer contact reasons had no limit on entries and grew beyond 1,400 options, most of them duplicates. Their assessment was that, at that point, the tags stop providing value and can cause more confusion than having none at all.

What actually works

Control	Detail
Restrict who can create	The single most effective control, and the reason organization-wide labels sit behind Org Admin permission. Where systems let any user create a tag without a permission check, overlapping and duplicated terms proliferate.
Two tiers, deliberately	A standard set that everything can use, plus local labels for genuinely local needs.
Start tight, loosen later	It is recommended that organizations begin with tight governance and then relax it as the vocabulary and the users mature. Tightening later means confiscating something people already use; loosening later is easy and welcomed.
A written one-page policy	Not a manual. A single page naming the owner, the request route to add a label at the Organization level, and the conventions.
Watch for trending topics	Subjects that keep coming up as a need across multiple Efforts. Every one of them tells you something the standard set is missing, which is more useful than any amount of guessing up front.

The counter-Intuitive part

Good governance here is not mostly about rules, it is about making the right thing easy. If picking an existing label is faster than typing a new one, the problem largely solves itself. If it is slower, no policy will hold, because the policy is competing with convenience and convenience wins every time.

Appendix E — Three Worked Examples

Three organizations, three industries, one method. In each case the phase list is written as parts of an Effort rather than stages work passes through, and in each case the most valuable phase turned out to be the one running alongside everything else.

E.1 A commercial fit-out contractor

Around 60 staff, running eight to twelve client sites at once, plus a bidding pipeline. Four Work Packages: Bids & Preconstruction, Active Projects, Safety & Compliance, and Plant & Fleet. Each individual site build is an Effort.

They took the six standard Agilic default states unchanged. The one they argued about was Blocked, because on a construction site almost everything is waiting for something. They kept it and made the Status note mandatory — which turned their weekly meeting into a list of eleven specific chases rather than a general complaint about delays.

Their Phases	What belongs in it
INITIATION	Opportunity spotted, go / no-go decision, client qualification.
ESTIMATE & BID	Take-off, pricing, subcontractor quotes, submission — and the wait for the client's decision.
MOBILIZE & PLAN	Site setup, permits, program, procurement, subcontractor appointments.
BUILD	Everything that happens on site to produce the work.
SAFETY & SITE ADMIN	Runs alongside. Toolbox talks, inspections, risk assessments, weekly client reports, variation paperwork.
SNAG & HANDOVER	Defects, client walkthrough, O&M manuals, certificates.
CLOSING	Retention released, final account, lessons captured.

Two decisions did the work here. ESTIMATE & BID holds the wait for a tender decision, which is months of real chasing that was previously invisible — and because Phases carry no dates of their own, that band simply stretches while they wait, which is exactly the signal they wanted.

SAFETY & SITE ADMIN is the parallel phase, and it was the surprise. It had always been absorbed into BUILD, which made BUILD look inexplicably expensive and made safety look free. Once it was its own phase, they could see it consuming roughly a fifth of every site week.

Their labels:

Kind	Labels
Unbound	Site (Site : Location 1, Site : Location 2) and Trade (Trade : Electrical, Trade : Heavy Equipment)
Bound	Priority (Priority :: High, Priority :: Low) and Client (Client :: A, Client :: B)
Simple	Rework, Weather Delay, Permit Required

Rework earns its place because it is the number they most want to reduce, and you cannot reduce what you cannot count.

E.2 A regional food-bank charity

Twelve staff and around 200 volunteers, funded by a mix of grants, donations and one local-authority contract. Four Work Packages: Programs, Fundraising & Grants, Volunteers, and Facilities & Logistics. Each funded program is an Effort.

They also took the six Agilic default states unchanged, but their argument was about Cancelled — several trustees wanted work simply deleted when dropped. They kept Cancelled specifically because funders ask why previously described activity did not happen, and a deleted item cannot answer that.

Their Phases	What belongs in it
INITIATION	Idea raised by staff, volunteers or the community. Mission fit assessed.
FUNDING	Identifying and securing the money. The hard gate, and largely outside their control.
PLANNING	Detailed design, rota, partnerships, safeguarding checks.
DELIVERY	Running the program.
REPORTING & COMPLIANCE	Runs alongside. Funder reports, outcome measurement, safeguarding reviews, volunteer records.
CLOSING	Signed off, reviewed, renewal decision made.

The important change was moving reporting out of the end and into a parallel phase. Their first attempt had REPORT sitting after DELIVERY, which is how everyone describes it and is not how it works — funder reporting happens throughout, which is precisely why it gets crowded out by delivery and why charities lose renewals. As a parallel phase it is visible during the program, when there is still time to do something about it.

Their labels:

Kind	Labels
Unbound	Funder (Funder : Source A, Funder : Source B) — a single program is often supported by multiple grants at once, and each wants its own report.
Bound	Service (Service :: Service 1, Service :: Service 2)
Simple	Grant Deliverable, Board Report

E.3 A software company shipping a release with an infrastructure build

A 20-person SaaS business. One Work Package, Release 4.0, containing two very different bodies of work: the application features, run in two-week sprints, and a migration from their own hardware to cloud infrastructure, which is sequential and dependency-heavy. They are in the same Effort because 4.0 cannot ship until the migration lands.

This is the case people assume needs two systems, or a choice between Agile and waterfall. In Agilic, it just needs a single Work Package, because Phases describe the parts of the Effort and both bodies of work are parts of the same Effort.

Their Phases	What belongs in it
INITIATION	Release scope agreed, business case for the migration, budget approved.
PLANNING	Target architecture, migration sequence, backlog shaping, cutover criteria.
APPLICATION DELIVERY	Runs alongside. All feature work, sprint by sprint. Which sprint an Item is in is carried by the sprint, not by the Phase.

Their Phases	What belongs in it
INFRASTRUCTURE BUILD	Runs alongside. Provisioning, data migration, parallel running, rollback preparation. Sequential internally, expressed as Item dependencies.
CUTOVER & RELEASE	The point where the two streams join. Go / no-go, switchover, release.
MONITORING & CONTROL	Runs alongside everything. Status reporting, change control, risk review, security sign-off.
CLOSING	Old hardware decommissioned, costs reconciled, retrospective held.

Three parallel phases in one Effort, and that is the entire point of the example. A model where work passes through one stage at a time would have forced them to declare one stream "behind" the other, which would have been meaningless — they are not the same work and they were never going to move at the same rate.

The Agile stream gets no phase per sprint. Sprints are cadence and Phases are structure, so every feature Item belongs to APPLICATION DELIVERY from creation to release regardless of which sprint carries it. Equally, the migration's internal steps are not phases; they are Items with dependencies between them.

Their labels:

Kind	Labels
Unbound	Component (Component : Area 1, Component : Area 2)
Bound	Priority (Priority :: Must Do, Priority :: Should Do)
Simple	Cutover Blocker, Rollback Required

The label they deliberately did not create

Their first draft had a Stream :: label with values App and Infrastructure. They dropped it, because the Phase already answers that question — and two fields answering one question is the failure this whole guide is built to prevent.

The reporting benefit showed up early. Because the two delivery phases have no dates of their own, their Gantt bands are drawn entirely from the Items inside them — so when APPLICATION DELIVERY began stretching past where INFRASTRUCTURE BUILD ended, the release date was visibly showing as at risk weeks before anyone would have known to escalate it.

What all three have in common

Every one of them found a phase that runs the whole way through and had never been named — site administration, funder reporting, monitoring and control. In each case that work had been absorbed into a delivery phase, which made the delivery look expensive and the invisible work look free. If you take one design instinct from this appendix, take that one: look for the work that never stops, and give it a name.

Appendix F — One-Page Checklist

Print this. Work through it with the two or three people who know how work actually moves in your organization.

✓	Step	Done when...
	Item States written	Six or fewer, none named after a kind of work, everyone can explain each in one sentence.
	Status expectation agreed	You have decided which states require a written reason. Blocked, On Hold and Cancelled is the usual answer.
	Phases written	Three to eight. Each one holds a body of work you can say "we finished this large section of the Effort" about.
	Parallel Phase identified	You have named the work that runs the whole way through — reporting, governance, safety, quality. Most organizations have one and most have never named it.
	Standard vs local Phases decided	You know which Phases most Work Packages use, which WP has an exception, and why.
	Teams decided	Your functional Team list is written, or you have consciously decided not to use Teams yet.
	Labels chosen	Each one has a named report or filter it feeds. None duplicates an existing field.
	Label kinds assigned	Every two-part label has been tested with: could one item ever honestly need two values?
	Label owner named	One person, not a committee. Everyone knows how to request a new label.
	Ten-item test run	Two people classified the same ten real items independently and agreed on at least nine of them.
	Disagreements resolved	Every difference between the testers has been traced to a definition and the wording fixed.

If you can tick all eleven, you have done this better than most organizations ever do. The next real test is six months from now: pull up a board and see whether the columns still mean what you wrote down here.

Appendix G — Terminology and Additional Training

Every term used in this guide, grouped by what it describes. If you are looking something up mid-task, the first table is almost certainly the one you want.

G.1 How work is structured

Term	What it means
Work Package (WP)	A body of work of any size. It can be a large multi-team project or a single person's small task — it is whatever size body of work you need to manage. One of the central concepts of Agilic.
WP Objective	The statement of what a Work Package is for. It sits at the top of the WP and everything inside it should serve it.
Effort	Informal shorthand used throughout this guide for the whole body of work a Work Package represents, from beginning to end.
Work Item Type	One of the containers a Work Package divides into. There are five: the four D's, plus RIDE.
Segment	The earlier name for a Work Item Type. Being retired — older material and some training videos still say Segment, so read it as Work Item Type.
The 4 D's	Define, Do Work, Document and Deliver — the four Work Item Types that hold delivery work. Define is working out what the thing is, Do Work is making it, Document is recording and verifying it, Deliver is handing it over. These are pre-defined; you do not design them.
Item	The lowest level of tracking in Agilic. Examples include an activity, a task, a list entry or a user story. Every Item sits inside a Work Item Type.
RIDE	Risks, Issues, Dependencies and Escalations — the fifth Work Item Type. Holds the things that are not delivery work but will stop delivery if ignored. Sometimes called Obtain: document it, show the Items it affects within the Work Package, then obtain something from someone, usually information, confirmation or a decision.
Work Breakdown Structure (WBS)	A hierarchical breakdown of the work, independent of WP Phases. You can align the two or not; both can be displayed on the Gantt.

G.2 Who does what

Term	What it means
Owner	Accountable for the success of a Work Package or Work Item Type. Accountability here is geared toward having the final say when situations arise.
Driver	Responsible for the delivery results of a Work Package or Work Item Type. Responsibility here is geared toward delivery of a thing.
Assigned	The person officially working on an Item.
Responsible	The point of contact for an Item. Not always the same person as the one Assigned.
Team	A functional group — Networking, Marketing, Electrical, Development. Individuals belong to a Team, and Team Views show all the work Team members are Assigned to. Useful for coverage when someone is out. See Step 3.

G.3 How work is classified

Term	What it means
Item State	Where a piece of work is right now — not started, moving, stuck, done. One list for the whole organization. See Step 1.
Item Status	Free text explaining WHY an Item is in its State. Automatically timestamped with who entered it. The most recent Status shows on the card.
WP Phase	Which part of the overall Effort an Item belongs to. Items normally stay in one Phase for their whole life. Phases are optional and can overlap. See Step 2.
Parallel Phase	A Phase that runs the entire length of the Effort alongside the others — typically reporting, governance, quality, safety or client communication. Most organizations have one and most have never named it.
Label	Any other classification. Three kinds: Simple (a flag or category), Unbound (category and value, several allowed per Item), Bound (category and value, exactly one per Item). See Step 4.
Organization-level label	Available across every Work Package. Only users with Org Admin permission can create one.
Work Package-level label	Created within a single WP for a need unique to that Effort.

G.4 Flags and views

Term	What it means
Milestone	A flag available on standard Items and on RIDEs, marking a checkpoint or moment in time.
Show on Roadmap	An additional flag on a Milestone, putting it on the roadmap views.
Escalated / Critical	Flags available on Milestones and RIDEs. Built in, so they never need a label.
Kanban	A board view. Item States, Phases and Labels can all be used as its columns, as can the Milestone and Show on Roadmap flags. Cards show the most recent Status, and Status can be updated without opening the Item.
Gantt	A timeline view. Phases appear as bands stretching from the earliest start date to the latest due date of the Items inside them, so they move as the work moves.

G.5 Additional Training

Resource	Where
General Training Videos	https://agilicsoftware.com/resources
Training Documents Wiki	Coming soon